

Pandas II

Funções essenciais

Ideia central: entender o problema, definir uma regra, transformar e conferir o resultado. Preserve sempre a base original.

1. `read_csv()` e `copy()` | carregar e preservar

`read_csv()` lê a tabela. `copy()` cria a cópia que receberá as alterações.

```
df_bruto = pd.read_csv("pacientes_aula5_bruto.csv")
df = df_bruto.copy()
```

Nos exemplos, `pd` vem de `import pandas as pd`, e `df` é a tabela de trabalho.

2. `isna()` e `sum()` | contar ausências

`isna()` identifica valores ausentes. `sum()` conta essas marcações em cada coluna.

```
df.isna().sum()
```

3. `duplicated()` e `drop_duplicates()` | repetições

`duplicated(keep=False)` mostra todas as ocorrências repetidas. `drop_duplicates()` remove duplicatas completas, mantendo a primeira.

```
df[df.duplicated(keep=False)]
df = df.drop_duplicates()
```

Cuidado: só remova após definir o que é duplicidade. Um paciente pode ter várias internações legítimas.

4. `str.strip()` e `str.lower()` | limpar texto

`strip()` remove espaços nas extremidades. `lower()` transforma letras em minúsculas.

```
df["servico_aux"] = df["servico"].str.strip().str.lower()
```

Exemplo: " **Cardiologia** " se torna "cardiologia". A coluna original permanece disponível para comparação.

Padronizar, converter e conferir

5. unique() e map() | padronizar categorias

unique() mostra os valores distintos. **map()** aplica um dicionário de correspondências. Valores fora do mapa ficam ausentes.

```
df["servico_aux"].unique()
mapa = {"cardiologia": "Cardiologia",
        "clínica medica": "Clínica Médica",
        "clínica médica": "Clínica Médica",
        "neurologia": "Neurologia"}
df["servico_padronizado"] = df["servico_aux"].map(mapa)
```

6. to_numeric() e to_datetime() | converter tipos

to_numeric() converte para números. **to_datetime()** converte para datas. Com **errors="coerce"**, valores não convertíveis viram ausentes.

```
df["idade_num"] = pd.to_numeric(
    df["idade"], errors="coerce"
)
df["data_alta_dt"] = pd.to_datetime(
    df["data_alta"], dayfirst=True, errors="coerce"
)
```

dayfirst=True prioriza o dia antes do mês. Confira o formato de origem, especialmente em datas ambíguas.

7. notna(), isna() e loc | verificar falhas

Localize valores presentes na coluna original que ficaram ausentes após a conversão. **loc** seleciona linhas e colunas; **&** significa "e".

```
falha = df["idade"].notna() & df["idade_num"].isna()
df.loc[falha, ["id_paciente", "idade"]]
```

8. to_csv() | salvar a base preparada

Depois de montar e conferir **df_limpo**, salve um novo arquivo. **index=False** evita incluir o índice como coluna extra.

```
df_limpo.to_csv("pacientes_aula5_limpo.csv", index=False)
```

Para lembrar: ausência não é zero. Uma conversão sem erro de execução ainda pode produzir valores que precisam de revisão.